UNIVERSITATEA TEHNICĂ
DIN CLUJ-NAPOCA

Faculty of Automation and Computer Science

Digital System Design
Automated Teller Machine

Custelcean Teodora-Yasmina

Danci Gabriella

Project supervisor: Ioana Virna

Table of contents

1. Project specifications
   1.1. Operating Procedure
   1.2. Safety Feature
2. Design
   2.1. Black Box
   2.2. Control and Execution Unit
      2.2.1. Mapping the inputs and the outputs of the black box on the 2 components CU and EU
      2.2.2. Establishing the resources (EU)
      2.2.3. State diagram of CU
      2.2.4. Detailed diagram of the project
3. User manual
4. Technical justification for the design
5. Further development
6. References

1. Project specifications

A8) Design a ATM for withdrawals in EURO. It is assumed that the maximum amount that can be extracted once is a maximum of 1,000 euros. Initially, the card is identified and the operation is chosen. At least 4 different cards / accounts will be supported and at least 4 different operations will be implemented. The vending machine has a cash register in which a certain amount is initially entered (number of banknotes of different values). In the case of a cash withdrawal, the amount is entered, the required amount is verified, the types of banknotes issued are displayed and the account is updated. Then the card, the amount and possibly the receipt are issued.

1.1. Operating Procedure

When powered on and reset, the ATM enters an idle state where it waits for the user to input a PIN code using a 4-bit input interface on the switches H6, U12, U11, V10. The PIN is validated against values stored in a RAM module dedicated to PIN codes. Upon pressing the confirmation button (btn_ok on N17), the system verifies the entered PIN. If the PIN is correct, led_ok (V11) is lit and the machine transitions to the operation selection state. If the PIN is incorrect, led_error (V16) is activated, and the system remains in the initial state.

Once authenticated, the user may choose one of four available operations: check balance (btn_check balance on M18), deposit money (btn_deposit on P17), withdraw money (btn_withdraw on M17), or change the PIN (btn_pin on P18). The user selects an operation by pressing the corresponding button. When checking the balance, the stored value is read from RAM, converted into individual digits, and displayed on a multi-digit 7-segment display. The digits are shown in a cycle, and the user may return to the initial state by pressing the confirmation button.

If the deposit operation is selected, the user enters the amount to deposit using a 7-bit input on switches from J15 to U18. Each bit corresponds to a predefined monetary value, and the total sum is computed internally. When the user confirms the operation, the new balance is calculated and stored, and the ATM's internal cash balance is updated accordingly. A receipt signal (receipt on led H17) is then issued and the system returns to the initial state.

The withdraw operation follows a similar process. The user specifies the withdrawal amount through the same 7-bit interface. The system checks whether the user’s account balance and the ATM's available funds are sufficient and smaller than 1000 euros. If the conditions are met, the new balance is computed and stored, and the ATM’s cash stock is reduced by the withdrawn amount. If the conditions are not satisfied, led_error is activated, and no withdrawal takes place. In either case, the user is returned to the PIN entry state.

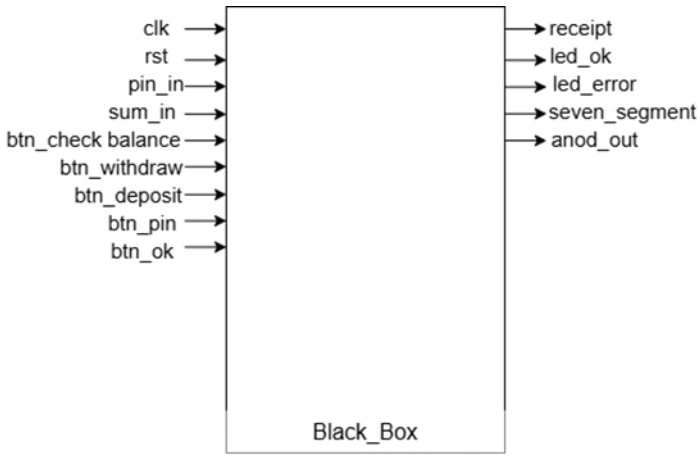
To change the PIN, the user selects the PIN change option (btn_pin) and enters a new 4-bit code. After confirmation, the new code is stored in the dedicated RAM. The system verifies the updated PIN, displays the account for which the pin was changed, issues a receipt, and resets to the initial state.

1.2 Safety Feature

The system validates user input to prevent unauthorized access. An incorrect PIN does not allow access to any operation and lights led_error to signal the failure. Additionally, before a withdrawal is processed, the system checks that the requested amount does not exceed the available balance in both the user's account and the ATM's internal fund register and is less than 1000 as requested. If this check fails, the system denies the transaction and indicates the issue via the error LED. Lastly, a reset function is provided that clears the system state, resets all internal signals and memory flags, and returns the ATM to the default idle state.

2. Design

2.1 Black Box

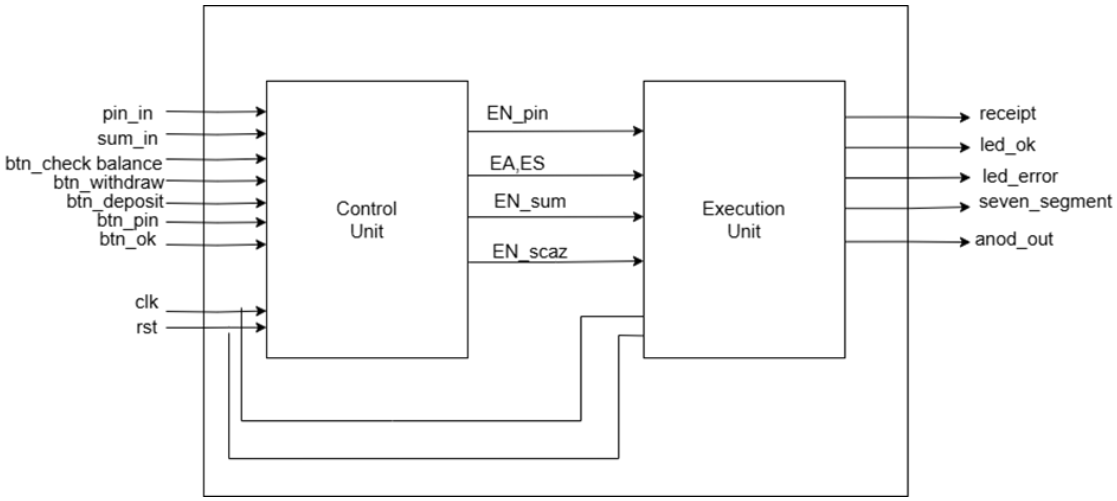


Following the synthesis process, we obtain the block diagram of the ATM that we want to implement. This encapsulates the internal processing of the ATM. It handles user authentication, processes transactions, and determines the appropriate response based on the input.

2.2 Control and Execution Unit

To identify implementable components within the system's black box, we begin by decomposing it through a top-down approach. This process continues until we reach familiar, well-defined circuits. Once those are identified, we proceed with a bottom-up implementation.

The first and most fundamental division within any system is between the control logic and the system resources. The control logic is encapsulated in the **Control Unit (CU)**, while the system resources are managed by the **Execution Unit (EU)**.



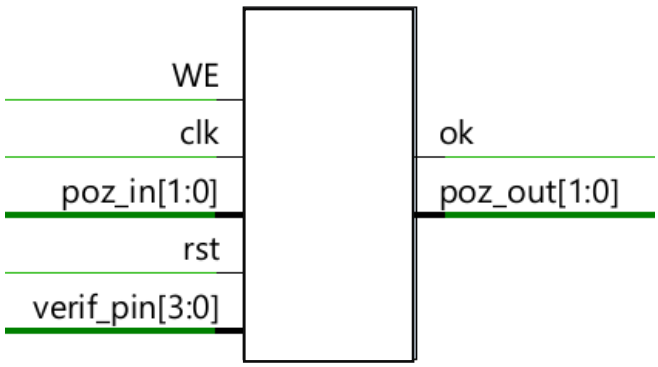
We can divide both inputs and outputs into 2 categories: data and control. This separation is essential at the beginning.

Data inputs: **pin_in**, **sum_in** – which allow us to read data from the switches
Control inputs: **clk**, **rst, btn_ok, btn_withdraw, btn_deposit, btn_check_balance, btn_pin**
Data outputs: seven_segment, anod_out – used to print out on our NexysA7100T seven segment display.
Control outputs: receipt, led_ok, led_error – all the control outputs are LEDs, used to indicate the succes of an action, sometimes an error and the receipt.

2.2.1. Resources

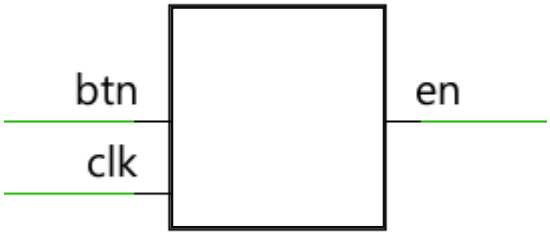
In order to further establish the links between the CU and the EU, we must first identify the resources on the basis of which we make decisions. These resources can generate signals to the control unit and can be controlled by the CU via Enable or Reset signal.

RAM_pin



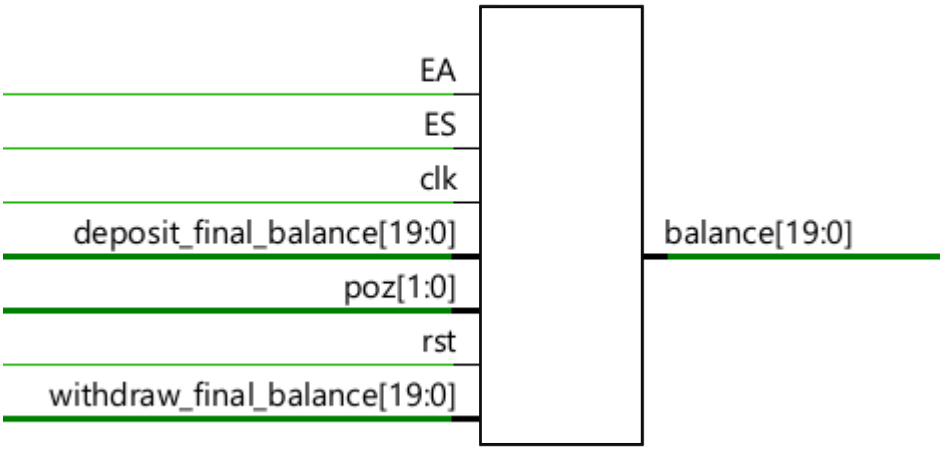
RAM_pin handles pin verification and storage. It stores 4 pin codes, verifies the entered pin and allows writing a new pin if it's not already stored.

Debouncer



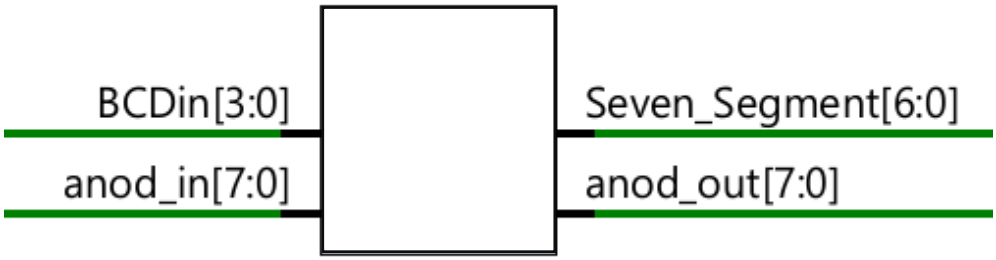
We use the debouncer to eliminate signal noise, ensuring that each button press is registered as a single, stable input. It prevents multiple false detections of a single button press.

RAM_sold



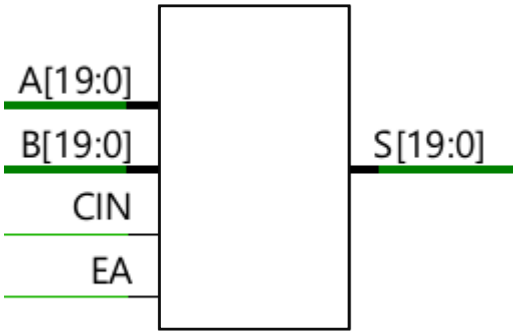
We use it to store, update balances on deposit/withdrawal, and output the current balances for up to 4 accounts (0 to 3).

Bcd7segment



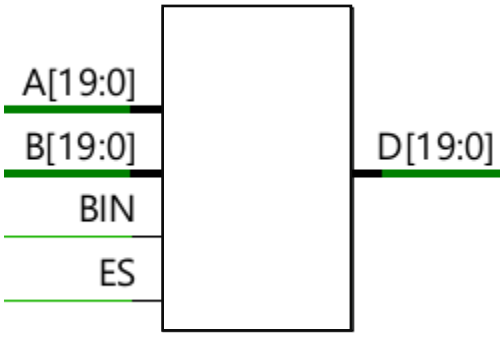
This component is for displaying numeric values (balance) on the 7-segment display. It translates a 4-bit number into segment control signals and helps manage which digit is currently being displayed using the anode control signals.

adder



This represents a 20-bit binary adder. We use it during deposits to add the new sum to the old sum, which updates the account balance.

Subtracter



This is a 20-bit binary subtractor. We use it during withdraw to subtract value B (withdrawal amount) from value A (current balance).

2.3. State diagram of Control Unit

The state diagram is **not a flow chart**, but represents the control part, the decision part of any algorithm, and it can then be implemented directly in VHDL if done correctly.

States are represented by

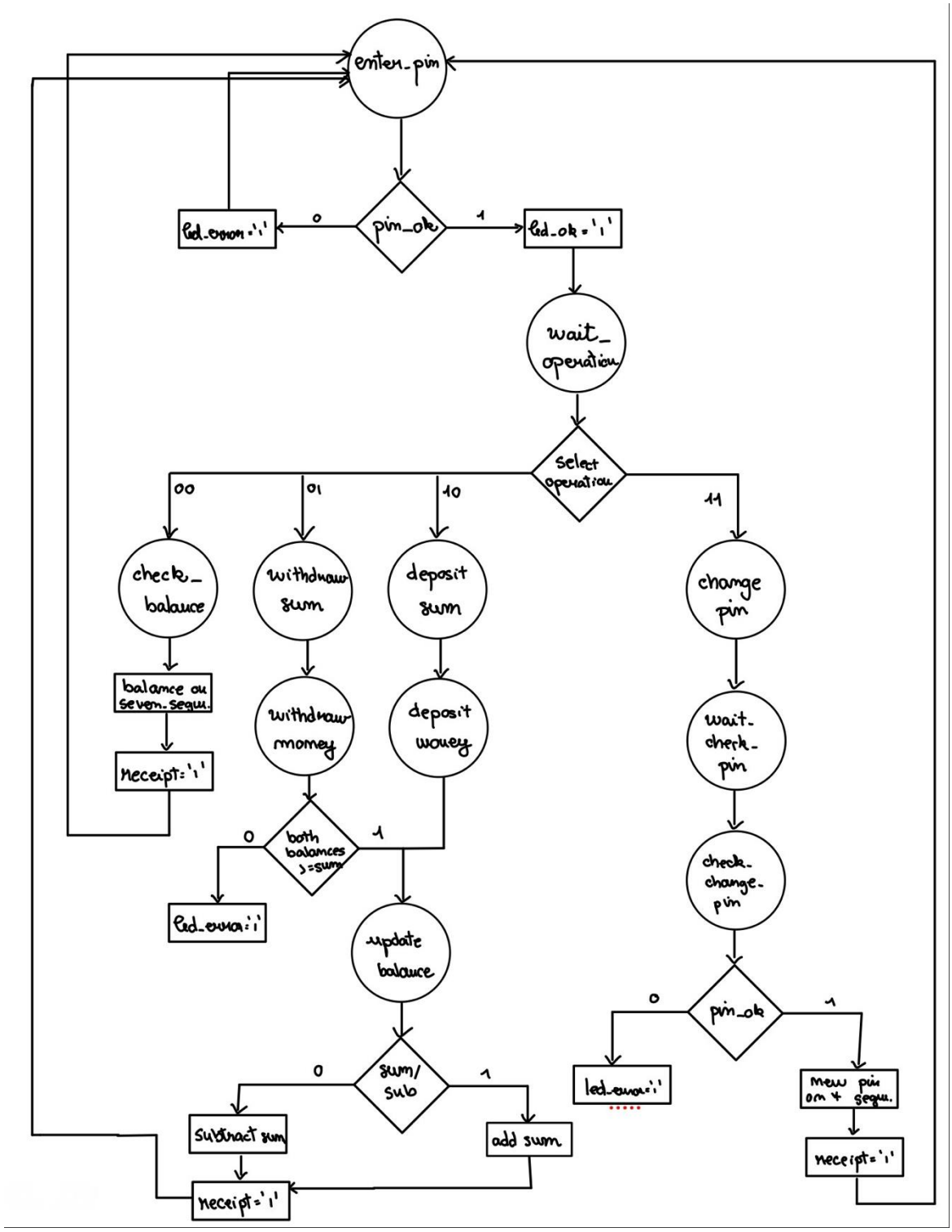
. A state represents a moment of time (a period).

The **decisions** made in each state are represented by

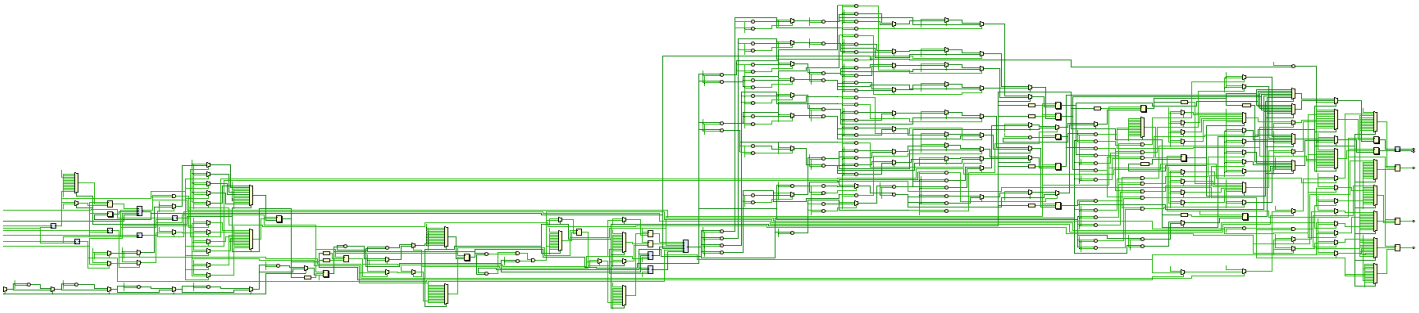
.

The outputs generated in each state are represented by led_ok='1'.

Inside the rectangles are the outputs that are true at that time. These outputs can be positioned *before* or *after* the decision. If they are *before* the decision, it means that the outputs depend only on the internal state,so we have a **Moore** automaton. If they are *after* the decision, it means that the output also depends on the internal stateand the inputs, so we have a **Mealy** automaton.


enter_pin
pin_ok
led_error='1'
0
1
led_ok='1'
wait_operation
select operation
00
01
10
11
check_balance
withdraw sum
deposit sum
change pin
balance on seven_segm.
withdraw money
deposit money
wait_check_pin
receipt='1'
both balances >=sum
check_change_pin
update balance
pin_ok
sum/sub
led_error='1'
new pin on 4 segm.
subtract sum
add sum
receipt='1'

2.2.4 Detailed diagram of the project



3. User Manual

This user manual explains, step by step, how the user should interact with the ATM system implemented on the FPGA. It includes the actions the user must take and the expected system behavior at each point.

When the system is powered on and reset, it initializes into the enter_pin state. At this moment, all control LEDs are off and the display is blank. The user must input a 4-bit PIN using the available switches and then press the confirmation button labeled OK (btn_ok). If the PIN is correct, the green LED (led_ok) lights up, and the machine enters the wait_operation state. If the PIN is incorrect, the red LED (led_error) is activated, and the machine remains in the enter_pin state, awaiting another attempt.

Once the PIN is successfully verified, the user is presented with four options. These are selected using dedicated push buttons: check balance, deposit money, withdraw money, and change PIN. When one of these buttons is pressed, the state machine transitions to the corresponding state.

If the user selects to check the balance, the current stored account value is read from memory and translated into six decimal digits, which are displayed on a multiplexed 7-segment display. The digits are shown sequentially, and the user may press the OK button to signal the end of this viewing session. This also activates the receipt signal and resets the state machine to the initial PIN entry state.

In the case of a deposit, the user inputs the amount to deposit using the 7-bit binary switches. Each bit corresponds to a specific denomination (1000, 500, 200, 100, 50, 20, 10). After selecting the desired combination of amounts, the user presses the OK button. The system

then computes the total sum, updates both the user's balance and the internal ATM balance, activates the receipt output, and resets to the PIN input state.

For withdrawals, the procedure is similar to the deposit. The user specifies the withdrawal amount using the same 7-bit interface. When the OK button is pressed, the system checks if the requested amount is less than or equal to both the user's balance and the ATM's internal cash store. If so, the withdrawal is performed, balances updated, and a receipt generated. If the check fails, the error LED is activated to inform the user of the problem, and the transaction is aborted.

Finally, for changing the PIN, the user presses the corresponding button, enters a new 4-bit PIN, and confirms with the OK button. The PIN is written into memory and verified. If successful, the user's ID is displayed, and the receipt signal is triggered.

Throughout all stages, feedback is provided via LEDs and the 7-segment display. The user can always start a new session by resetting the system or re-entering their PIN after the receipt is generated. This design ensures that all interactions are intuitive, secure, and visibly confirmed.

4.Design Solutions Justifications

The ATM system was designed with a clear separation between control logic and data processing, using a finite state machine (FSM) for control and discrete functional blocks for data handling. This modular architecture was chosen to improve the clarity and maintainability of the system.

Use of Finite State Machine (FSM)

The control logic is implemented using a finite state machine that explicitly defines all the operational phases: PIN entry, operation selection, deposit/withdrawal handling, balance checking, and PIN update. An FSM was preferred over hard-coded combinational logic because it allows for predictable sequencing of events, clear state-based transitions, and easy debugging. Alternative designs could have used combinational logic with priority-encoded decision trees, but this would have made the design significantly harder to follow.

The FSM uses symbolic state names and transitions driven by debounced input signals and internal flags, making sure the system works correctly even if the inputs are asynchronous.

RAM-Based Storage for PIN and Balance

We opted to use RAM components for both the user PIN (RAM_pin) and account balance (RAM_sold). An alternative would have been to use registers for each user's data.

However, RAM allows scalable storage for multiple users, and its dual role as both read and write memory simplifies PIN changes and balance updates. The RAM interface also enables indexing users by position, which is essential for a multi-user ATM system simulation.

Separate Adder and Subtracter Components

Rather than using a single arithmetic unit with control signals to switch between addition and subtraction, we implemented separate adder and subtracter components. This design choice simplifies logic in each arithmetic block and avoids introducing additional multiplexing logic or control overhead. It also makes debugging easier by isolating each operation. Although sharing arithmetic hardware could reduce resource usage slightly, the chosen approach favors clarity.

BCD to 7-Segment Display with Multiplexing

The balance is shown using a 6-digit 7-segment display system with BCD encoding and digit multiplexing. This was selected because it provides a readable numeric output for users while maintaining compatibility with standard FPGA board displays. Instead of displaying a binary number directly, which is non-intuitive for users, the decimal conversion via BCD improves usability. An alternative would have been to use an external display driver chip or a UART-based text display, but these would require additional hardware interfaces and would not be directly supported on typical FPGA boards.

Debounced Input Handling

Mechanical switches are prone to bouncing, which can trigger multiple unintended transitions. For this reason, a debouncer component was included for all user input buttons. This decision ensures clean transitions and stable behavior in state logic. Omitting debouncing would have led to erratic behavior, especially in fast-transitioning states like deposit or withdrawal confirmation.

Explicit ATM Cash Tracking

The design maintains a separate register for the ATM's available cash (ATM_balance), which is updated alongside user balance operations. This prevents over-withdrawals and models real-world ATM limitations. While this could have been omitted for a simplified design, we included it to enhance realism and enforce dual-condition checks (user balance and ATM cash) during withdrawal. The decision helps demonstrate safe transactional logic in digital systems.

Reset and Safety Indicators

The system includes a reset input to return all logic and memory to a known safe state. It also uses two status LEDs to indicate successful or failed operations. This approach provides immediate visual feedback to users and testers. Alternative designs could use serial or graphical displays for diagnostics, but LEDs are more suitable for embedded FPGA contexts where simplicity and visibility are important.

5. Further Development

The current version of the ATM system implements a basic set of banking operations including PIN verification, balance inquiry, deposit, withdrawal, and PIN update. While functionally complete for a simple simulation, the application offers several opportunities for further development to enhance its realism, usability, and scalability.

One possible extension would be the implementation of a **multi-user login system with user identification via card IDs or unique account numbers**. In the current design, users are implicitly indexed and share the same interface. By adding a numeric account ID input (e.g., through switches or a keypad), each user could be uniquely identified before entering their PIN. This would allow support for many users in a more realistic way and improve security by combining ID and PIN authentication.

A second enhancement could be the **integration of a transaction log** that records all deposit, withdrawal, and PIN change events. This could be implemented using a small FIFO or RAM buffer that stores the last few transactions. It could display the most recent actions on request via the 7-segment display or be transmitted externally through a serial interface such as UART. This addition would make the ATM behave more like a real banking system, enabling auditing and debugging of user activity.

Another development path involves **enhancing the user interface** by adding a **touch-based or keypad menu system**that displays operation prompts and user instructions. While the current system relies on buttons and LEDs, a text-based display module (like an LCD output) could provide more intuitive feedback, reducing the need for external documentation.

These proposed features demonstrate how the current ATM implementation can evolve from a training-oriented digital design into a more complete and interactive embedded application.